

Chapter Five

Exception handling overview

Abey B.(MSc)

**Debre Berhan University(DBU)
Faculty of Computing
Department Of Software Engineering**

January 23, 2025

What is an exception?

- **Exception** is an event that occurs during the **execution of a program that disrupts the normal flow of instructions.**
- **An exception is an error** that occurs **during the execution of a program**
- **An Exception is an unwanted or unexpected** event that occurs **during the execution of a program** (i.e., at runtime) and **disrupts the normal flow of the program's instructions.**

What is an exception?

- **An exception in Java** is an **object representing** an **error or an abnormal condition** that occurs at **runtime execution** and **interrupts (disrupts)** the **normal execution flow of the program**.
- **Unwanted** and **unexpected behavior**/event that **interrupts the normal execution** flow of the program is called **exception in Java**.
- **An exception** can be **identified** only **at runtime, not at compile time**.

What is an exception?

Realtime Example of Exception in Java

- Suppose you are watching a video on Youtube, suddenly, internet connectivity is disconnected or not working. In this case, you are not able to continue watching the video on Youtube. This interruption is nothing but an exception.



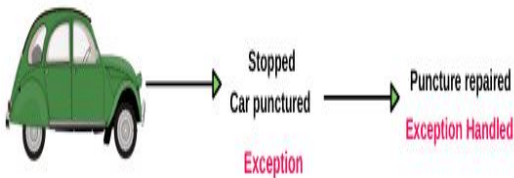
What is an exception handling?

- The mechanism of **handling unexpected errors** in a Java program is called **exception handling**.
- **Exception handling** is the process of **responding to unwanted or unexpected events** when a **computer program runs**.
- **Exception handling** deals with these events to **avoid the program or system crashing**, and **without this process**, exceptions would **disrupt the normal operation of a program**.
- **Exception handling** in Java is an effective mechanism for managing runtime errors to ensure the application's regular flow is maintained.

What is an exception handling ?

Realtime Example of Exception in Java

- Suppose a person is traveling by car from X to Y. After traveling mid-distance, the tire of his car is punctured. This unexpected or unwanted event is nothing but an exception.
- The car owner always keeps an extra tire as an alternative on a long-distance journey. He changes the punctured tire with a new tire. After changing the tire, he continues the rest of the journey. This alternative way is called exception handling.



The causes of exception?

- **Scenario** where **ArithmeticException** occurs
 - ▶ If we **divide any number by zero**, there occurs an **ArithmeticException**.
 - ▶ Example `int a=50/0;//ArithmeticException`

The causes of exception?

- **Scenario** where **ArithmeticException** occurs

```
class Main {  
    public static void main(String[] args) {  
        int divideByZero = 5 / 0;  
        System.out.println(divideByZero);  
    }  
}
```

- **OutPut**

Exception in thread "main" java.lang.ArithmeticException: / by zero
at Main.main(Y.java:3)

The causes of exception?

- **Scenario where NullPointerException occurs**

- ▶ If we have **null value** in **any variable**, performing any operation by the **variable** occurs a **NullPointerException**.

- ▶ **Example:**

String s=null

System.out.println(s.length());//NullPointerException

The causes of exception?

- **Scenario where NumberFormatException occurs**
 - ▶ The wrong formatting of any value, may occur **NumberFormatException**.
 - ▶ Suppose I have a **string variable** that have characters, **converting this variable into digit** will occur **NumberFormatException**.

The causes of exception?

- **Scenario where `ArrayIndexOutOfBoundsException` occurs**

- ▶ If you are inserting any value in the wrong index, it would result `ArrayIndexOutOfBoundsException` as shown below:

- ▶ **Example:**

```
int a[]=new int[5];  
a[10]=50; //ArrayIndexOutOfBoundsException
```

The causes of exception?

- Scenario where **ArrayIndexOutOfBoundsException** occurs

```
public class Main {  
    public static void main(String[ ] args) {  
        int[] myNumbers = {1, 2, 3};  
        System.out.println(myNumbers[10]);  
    }  
}
```

- **Output**

Exception in thread "main"

java.lang.**ArrayIndexOutOfBoundsException**: 10 at
Main.main(Main.java:4)

Types of exceptions?

① Checked exceptions.

- ▶ Also called **compile-time exceptions**
- ▶ The compiler **checks** these exceptions **during the compilation process** to confirm if the exception is being **handled by the programmer**.
- ▶ If not, then a compilation error displays on the system. Checked exceptions include **SQLException** and **ClassNotFoundException**.

Checked exceptions

Examples of Checked Exception are listed below:

- **ClassNotFoundException**

- ▶ When the program **tries to load a class at runtime** but the **class is not found** because **its not present in the correct location or it is missing from the project.**

- **InterruptedException**

- ▶ When a **thread is paused** and **another thread interrupts it.**

- **IOException**

- ▶ When **input/output operation fails**

- **SQLException**

- ▶ When there's **an error with the database.**

- **FileNotFoundException**

- ▶ When the program tries to **open a file that doesn't exist**

Types of exceptions?

① Unchecked exceptions.

- ▶ Also called **runtime exceptions**, these exceptions occur during **program execution**.
- ▶ These exceptions are **not checked at compile time**, so the programmer is responsible for handling these exceptions.
- ▶ **Examples** of unchecked exceptions include **NullPointerException** and **IllegalArgumentException**.

Java exception keywords

- **Java provides five keywords** that are used to **handle the exception**. Those are:

- 1 **try**
- 2 **catch**
- 3 **finally**
- 4 **throw**
- 5 **throws**

Java try block

- The **"try"** keyword is used to **specify a block** where we **should** place an **exception code**.
- The **"try"** keyword is used to **enclose the code that might throw** an **exception**.
- We can't use **try block alone**
- It is **not recommended** to keep the code in **try block** that **will not** throw an **exception**.
- The **try block** must be followed by **either catch or finally**

Try....Catch Block

- A **try...catch block** in Java is a mechanism to **handle exception**.
- The **try block** contains code that **might throw an exception** and the **catch block** is used to **handles the exceptions if it occurs**.

```
try {  
    // Code that may throw an exception  
  
} catch (ExceptionType e) {  
    // Code to handle the exception  
}
```

Try....Catch Block

- Case 1:

- ▶ Suppose no exception occurs inside try block

```
try {  
    statement 1;  
    statement 2;  
    statement 3;  
  
} catch (ExceptionType object) {  
  
    statement 4;  
  
}  
statement 5;
```

Try....Catch Block

- **Case 1:**

- ▶ Suppose no exception occurs inside try block

```
try {  
    statement 1;  
    statement 2;  
    statement 3;  
  
} catch (ExceptionType object) {  
    statement 4;  
}  
statement 5;
```

- **Output is:**

statement1
statement2
statement3
statement5;

- **Termination** of the program is **Normal**

Try....Catch Block



```
class Main {
    public static void main(String[] args) {
        int num[]={7,1,12,19,16,19};
        try{
            System.out.println("num[0]      "+num[0]);
            System.out.println("num[1]      "+num[1]);
            System.out.println("num[2]      "+num[2]);
            System.out.println("num[3]      "+num[3]);
            System.out.println("num[4]      "+num[4]);

        }
        catch(ArrayIndexOutOfBoundsException a)
        {
            System.out.println("Array out of the bound")

        }
        System.out.println("num[5]      "+num[5]);
    }
}
```

Try....Catch Block

- **Output of the above example**

num[0] 7

num[1] 1

num[2] 12

num[3] 19

num[4] 16

num[5] 19

Try....Catch Block

- Case 2:
 - ▶ Suppose exception occurs in statement 2 inside try block

```
try {  
    statement 1;  
    statement 2;  
    statement 3;  
  
} catch (ExceptionType object) {  
  
    statement 4;  
  
}  
statement 5;
```

Try....Catch Block

- **Case 2:**
 - ▶ Suppose no exception occurs inside try block

```
try {  
    statement 1;  
    statement 2;  
    statement 3;  
  
} catch (ExceptionType object) {  
    statement 4;  
}  
statement 5;
```

- **Output is:**
statement1
statement4
statement5
- **Termination** of the program is **Normal**

Try....Catch Block



```
class Main {
    public static void main(String[] args) {
        int num[]={7,1,12,19,16,19};
        try{
            System.out.println("num[0]      "+num[0]);
            System.out.println("num[6]      "+num[6]);
            System.out.println("num[2]      "+num[2]);
            System.out.println("num[3]      "+num[3]);
            System.out.println("num[4]      "+num[4]);

        }
        catch(ArrayIndexOutOfBoundsException a)
        {
            System.out.println("Array out of the bound")

        }
        System.out.println("num[5]      "+num[5]);
    }
}
```

Try....Catch Block

- **Output of the above example**

num[0] 7

Array out of the bound

num[5] 19

Try....Catch Block

- Case 3:

- ▶ Suppose exception occurs in statement 2 and expectation object created is not matched with argument of the catch block

```
try {  
    statement 1;  
    statement 2;  
    statement 3;  
  
} catch (ExceptionType object) {  
    statement 4;  
}  
statement 5;
```

Try....Catch Block

- Case 3:
 - ▶ Suppose no exception occurs inside try block

```
try {  
    statement 1;  
    statement 2;  
    statement 3;  
  
} catch (ExceptionType object) {  
    statement 4;  
}  
statement 5;
```

- **Output is:**
statement1
- Termination of the program is **Abnormal Termination**

Try....Catch Block



```
class Main {
    public static void main(String[] args) {
        int num[]={7,1,12,19,16,19};
        try{
            System.out.println("num[0]      "+num[0]);
            System.out.println("num[6]      "+num[6]);
            System.out.println("num[2]      "+num[2]);
            System.out.println("num[3]      "+num[3]);
            System.out.println("num[4]      "+num[4]);

        }
        catch(ArithmeticException a)
        {
            System.out.println("Array out of the bound")
        }
        System.out.println("num[5]      "+num[5]);
    }
}
```

Try....Catch Block

- **Output of the above example**

num[0] 7

Exception in thread "main"

java.lang.ArrayIndexOutOfBoundsException: Index 6 out of bounds
for length 6

Try....Catch Block

- Case 4:
 - ▶ Suppose exception occurs in statement 5

```
try {  
    statement 1;  
    statement 2;  
    statement 3;  
  
} catch (ExceptionType object) {  
  
    statement 4;  
  
}  
statement 5;
```

Try....Catch Block

- **Case 3:**
 - ▶ Suppose no exception occurs inside try block

```
try {  
    statement 1;  
    statement 2;  
    statement 3;  
  
} catch (ExceptionType object) {  
    statement 4;  
}  
statement 5;
```

- **Output is:**
statement1
statement2
statement3
- **Termination** of the program is **Abnormal Termination**

Try....Catch Block



```
class Main {
    public static void main(String[] args) {
        int num[]={7,1,12,19,16,19};
        try{
            System.out.println("num[0]    "+num[0]);
            System.out.println("num[1]    "+num[1]);
            System.out.println("num[2]    "+num[2]);
            System.out.println("num[3]    "+num[3]);
            System.out.println("num[4]    "+num[4]);

        }
        catch(ArithmeticException a)
        {
            System.out.println("Array out of the bound")
        }
        System.out.println("num[6]    "+num[6]);
    }
}
```

Try....Catch Block

- **Output of the above example**

num[0] 7

num[1] 1

num[2] 12

num[3] 19

num[4] 16

Exception in thread "main"

java.lang.ArrayIndexOutOfBoundsException: Index 6 out of bounds
for length 6

Try....Catch Block

```
class Main {  
    public static void main(String[] args) {  
        try  
        {  
            int data=50/0;  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println(e);  
        }  
        System.out.println("rest of the code");  
    }  
}
```

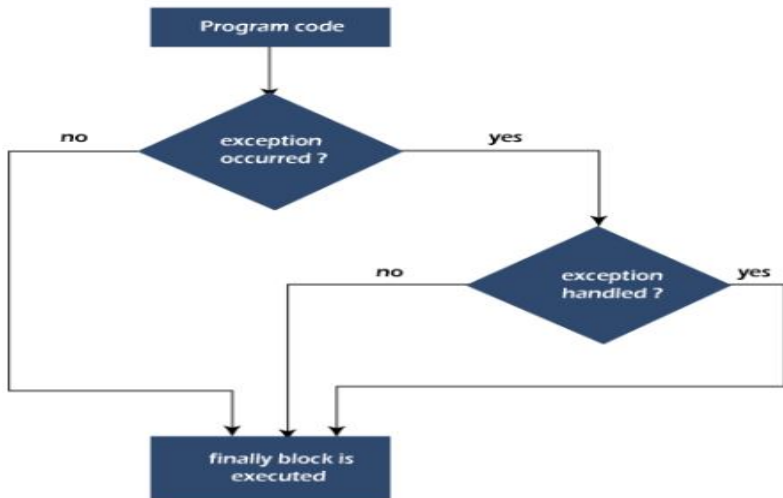
Try....Catch Block

- **Output of the above example**
java.lang.ArithmeticException: / by zero
rest of the code

finally Block

- The **finally Block** is used to **execute important code** regardless of whether an exception occurs or not.
- The **"finally" block** is used to **execute the necessary code** of the program.
- **finally block** is always executes after the try-catch block.
- **Note:** If you don't handle the exception, before terminating the program, **JVM** executes finally block (if any).

finally Block



finally Block

Why use Java finally block?

- ① **finally block in Java** can be used to **put "cleanup" code** such as **closing a file, closing connection, etc.**
- ② **The important statements to be printed** can be placed **in the finally block.**

finally

Syntax

```
try {  
    // Protected code  
} catch (ExceptionType1 e1) {  
    // Catch block  
} catch (ExceptionType2 e2) {  
    // Catch block  
} catch (ExceptionType3 e3) {  
    // Catch block  
} finally {  
    // The finally block always executes.  
}
```


Usage of Java finally

Case 1:

- Where the Java program **does not throw any exception**, and the **finally block is executed after the try block**.

```
class TestFinallyBlock {  
    public static void main(String args[]){  
        try{  
            int d=25/5;  
            System.out.println(d);  
        }  
        catch(NullPointerException e){  
            System.out.println(e);  
        }  
        finally {  
            System.out.println("finally block is always executed");  
        }  
  
        System.out.println("rest of code...");  
    }  
}
```

Usage of Java finally

Case : 1

- **Output of the above program**

5

finally block is always executed
rest of code...

Usage of Java finally

Case 2:

- When an **exception occurred** but **not handled by the catch block**

```
public class TestFinallyBlock1{
    public static void main(String args[]){
        try {
            System.out.println("Inside the try block");
            int d=25/0;
            System.out.println(d);    }
        catch(NullPointerException e){
            System.out.println(e);    }
        finally {
            System.out.println("finally block is always executed");
        }
        System.out.println("rest of the code...");
    } }
```

Usage of Java finally

Case 2:

- **Output of the above program**

Inside the try block

finally block is always executed

Exception in thread "main" java.lang.ArithmeticException: / by zero

Usage of Java finally

Case 3:

- When **an exception occurs** and is **handled by the catch block**

```
class TestFinallyBlock2{
    public static void main(String args[]){
        try {

            System.out.println("Inside try block");
            int d=25/0;
            System.out.println(d);    }
        catch(ArithmeticException e){
            System.out.println("Exception handled");
            System.out.println(e);    }
        finally {
            System.out.println("finally block is always executed");
        }

        System.out.println("rest of the code...");
    } }
```

Usage of Java finally

Case 3:

- **Output of the above program**

Inside try block

Exception handled

java.lang.ArithmeticException: / by zero

finally block is always executed rest of the code...

Try....finally

```
class Main {  
    public static void main(String[] args) {  
        try {  
            int divideByZero = 5 / 0;  
        }  
  
        finally {  
            System.out.println("Finally block is always executed");  
        }  
    }  
}
```

try...finally

- **Output of the above program**

Finally block is always executed

Exception in thread "main" java.lang.ArithmeticException: / by zero

Try.

```
import java.util.*;
class Geeks {
    public static void main(String[] args) {
        try {
            int res = 10 / 0;
        }
        catch (ArithmeticException e) {
            System.out.println(
                "Caught ArithmeticException: " + e);
        }
        catch (NullPointerException e) {
            System.out.println(
                "Caught NullPointerException: " + e);
        }
    }
}
```

Class Work

- **Write program that accept two numbers and division of the two numbers using exceptional handling**

Class Work

- **Write program that accept two numbers and division of the two numbers using exceptional handling**